FUNCTIONAL RISK MINIMIZATION

$\Omega = X \times Y$, $f: X \to Y$, $\ell: Y \times Y \to \mathbb{R}_+$, μ

(*) $L(f) = \int_\Omega \ell(f(x), y)\, d\mu(x,y)$ ← FUNCTIONAL RISK

$\mathrm{dom}(L)$ is the domain in wich we define L

SML (Vapnik "Statistical Learning Theory")

Find a model f as a solution of the min prob.

$$\min \{ L(f) : f \in \mathrm{dom}(L) \}. \quad (1) \qquad f^*$$

1. Availability of μ (The proposed solution was to approximate L with L_N ← Empirical Risk)

Now one can ask if it make sense to solve

$$\min \{ L_N(f) : f \in \mathrm{dom}(L) \}. \quad (2)$$

What is the Empirical Risk?

The Empirical risk is obtained from the Functional Risk if we replace μ with

$$\mu_N := \frac{1}{N} \sum_{i=1}^{N} \delta_{(x_i, y_i)}$$

, where $T = (x_i, y_i)_{i=1}^{N}$ is called the training set

and $\delta_{(x_i,y_i)}$ is the Dirac Measure centered in (x_i, y_i)

We assume (x_i, y_i) are iid and drawned from μ

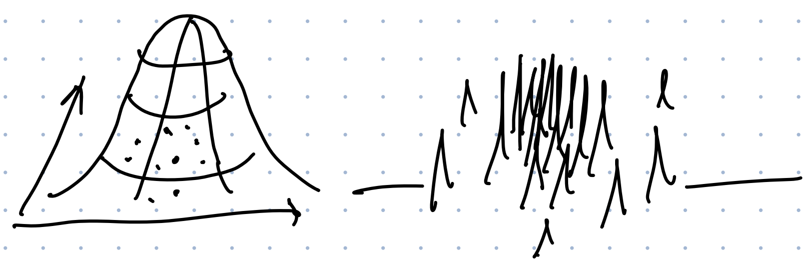
If you replace μ with μ_N in (*)

$$L_N(f) = \frac{1}{N} \sum_{i=1}^{N} \ell(f(x_i), y_i)$$

L_N hopefully approximate L

$$L(f) = \int_\Omega \ell(f(x), y)\, d\mu(x,y)$$

→ This has been replaced by $\frac{1}{N} \sum_{i=1}^{N} d\delta_{(x_i, y_i)}$



2. $f \in \text{dom}(L)$ is not so obvious how to generate.

Restrict the minimization to a parametric family of models

$$M = \{ f(\cdot, w) : X \to Y \ : \ w \in \mathbb{R}^n \}.$$

Example $\quad X = Y = \mathbb{R} \qquad X \times Y = \mathbb{R}^2$

$$M = \{ f(x, w) = w_1 x + w_2 \ , x \in \mathbb{R} : (w_1, w_2) \in \mathbb{R}^2 \}.$$

We will be interested in considering M to be the class of NN with some fixed architecture.

So now it make sense to consider the following min prob

$$\min \{ L_N(f(\cdot; w)) \ : \ w \in \mathbb{R}^n \}. \qquad (3)$$

$$w^*$$

$$\| L(f^*) - L_N(f(\cdot; w^*)) \| \leq ① + ②$$

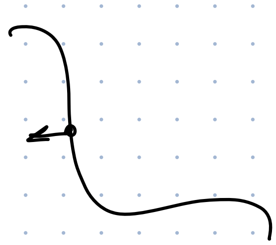
GRADIENT DESCENT

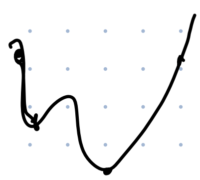
$f : \mathbb{R}^n \to \mathbb{R} \qquad f \in C^1$

$\min \{ f(x) : x \in \mathbb{R}^n \}$

∇f

1. $x_0 = x^0$
2. $x_{k+1} = x_k - b \nabla f(x_k) \qquad b > 0$





Algorithm 0

1. $x_0 = x^0$
2. $x_{k+1} \in \text{argmin} \{ f(x) : x \in \mathbb{R}^n \}.$

Algorithm 1

1. $x_0 = x^0$

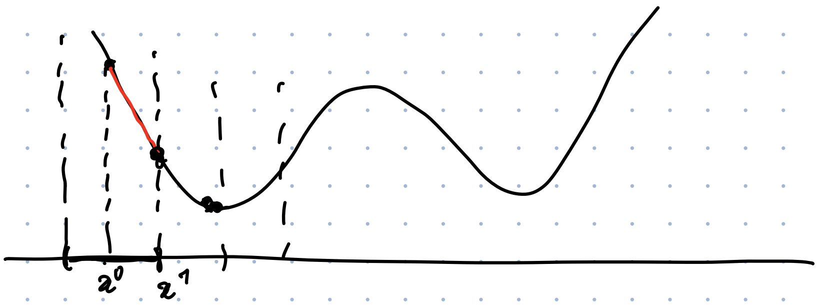
2. $x_{k+1} \in \operatorname{argmin} \{ f(x) : \|x - x_k\| \le \varepsilon \}$.

Algorithm 2

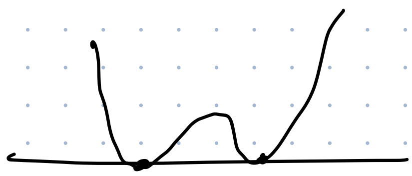
1. $x_0 = x^0$

2. $x_{k+1} \in \operatorname{argmin} \{ f(x) + \frac{1}{2\tau} \|x - x_k\|^2 : x \in \mathbb{R}^n \}$.

$k > 0$



Statement: Algorithm 2 is an implicit gradient descent if $f \in C^1$



10^{13}

$$\nabla \left(f(x) + \frac{1}{2t} \|x - x_k\|^2 \right) \Big|_{x = x_{k+1}} = 0$$

$$\nabla f(x_{k+1}) + \frac{1}{t}(x_{k+1} - x_k) = 0 \Rightarrow \boxed{x_{k+1} = x_k - t \nabla f(x_{k+1})}$$

$\dot{x}(t) = -\nabla f(x(t))$ Gradient flow on f.

$$\frac{x(t+\tau) - x(t)}{\tau} = -\nabla f(x(t))$$

$$-\nabla f(x(t+\tau))$$

Idea: Since we are optimizing f near x_k we can expand f around x_k

$$f(x) = f(x_k) + \nabla f(x_k) \cdot (x - x_k) + o(\|x - x_k\|)$$

Algorithm 3

$$x_0 = x^0$$

$$(x - x_k)\, H\, (x - x_k)$$

$$x_{k+1} \in \operatorname{argmin} \left\{ \nabla f(x_k) \cdot (x - x_k) + \frac{1}{2\tau} \|x - x_k\|^2 \; : \; x \in \mathbb{R}^n \right\}.$$

Exercise: Algorithm 3 is an explicit gradient descent of f with learning rate τ.

$$x_{k+1} = x_k - \tau \nabla f(x_k)$$

$$\dot{x} = -\nabla f$$

$$\underline{m}\,\ddot{x} + \dot{x} + \nabla f = 0$$

Gradient Descent with momentum

$$x_{k+1} \in \operatorname{argmin} \left\{ \nabla f(x_k) \cdot (x - x_k) + \frac{1}{2\mu} \|x - x_k\|^2 + \frac{1}{2\gamma} \|x - 2x_k + x_{k-1}\|^2 : x \in \mathbb{R}^n \right\}.$$

$$\|x - x_k - (x_k - x_{k-1})\|^2$$

$$x_{k+1} = x_k - \tau \nabla f(x_k) + \beta (x_k - x_{k-1})$$

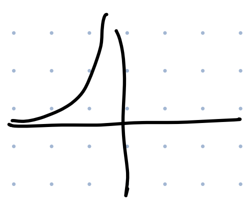
$$\beta = 0.9$$

$$\ddot{x} + \dot{x} + \nabla f = 0$$

$$\nabla f(x_k) \to 0$$

$$\dot{x}(t) = -\nabla f(x(t))$$

∇f should be L-Lipschitz



$$\tau < \frac{2}{L}$$

g is Lipschitz if

$$\|g(x) - g(y)\| \leq L \|x - y\|$$

$$f = \frac{1}{N}\sum_{i=1}^{N} f_i$$

$$x_{k+1} = x_k - \tau \nabla f_{I_k}(x_k)$$

I_k is a r.v. sampled on $\{1, \ldots, n\}$ in an iid manner